DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM

FOR CAR RENTAL RESERVATION AND VEHICLE RETURN MANAGEMENT SYSTEM

A Database Management Systems Project Report

Submitted in partial fulfilment of the requirements for the
Project-Based Learning (PBL) component of Database Management Systems

Project No. 19

BANGLA DEEKSHAN RAJ
25WU0102037
AIML RHINOS
WOXSEN UNIVERSITY

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Existing System

6. Proposed System

7. Methodology

8. System / Architecture Design

9. Implementation

10. Results / Output

11. Advantages

12. Limitations

13. Future Scope

14. Conclusion

15. References

(Note: exact page numbers are visible once opened in Microsoft Word.)

1. ABSTRACT

Car-rental agencies traditionally rely on registers or disconnected spreadsheets to track vehicle availability, customer bookings, pickup/return inspections and billing. This causes double-booking, disputes over vehicle condition, and delayed revenue reporting. This project designs and implements a relational database, normalized up to Third Normal Form (3NF), for a Car Rental Reservation and Vehicle Return Management System. The system models branches, vehicle categories, vehicles, customers, reservations, rentals, inspections, fuel readings, damages, rate plans, invoices and payments, and enforces business rules such as no overlapping rentals, valid driving licence, return-after-pickup and non-negative charges through primary/foreign keys, CHECK constraints and triggers. A Python (Tkinter) front-end application connects to the MySQL database and supports availability search, reservation, vehicle issue and return with inspection, automatic charge calculation, damage recording, payment collection and management reports such as fleet availability, overdue rentals, utilization, damages, branch revenue and customer history. The project demonstrates the complete database development life-cycle: requirement analysis, ER modelling, normalization, constraint-based integrity enforcement, query design and application integration.

2. INTRODUCTION

A car-rental agency's daily operations revolve around knowing, at any moment, which vehicles are free, which are booked, and which are out on rent. It must also record the condition of a vehicle before and after every rental so that damage disputes can be resolved fairly, and it must bill customers correctly for the days used, the distance travelled and any late return. When this information is kept manually or in isolated spreadsheets at each branch, the same vehicle can be promised to two different customers, damage claims become 'his word against ours', and revenue figures take days to compile across branches.

A relational database management system solves this by keeping one consistent, rule-enforcing source of truth that every branch and every part of the application reads from and writes to. This project applies the standard database design process — requirement analysis, conceptual (ER) modelling, logical design and normalization, physical implementation with constraints, and application development — to build a small but complete Car Rental Reservation and Vehicle Return Management System.

3. PROBLEM STATEMENT

Design and implement a relational database management system for a Car Rental Reservation and Vehicle Return Management System that:

- Maintains reliable records of branches, vehicle categories, vehicles, customers, reservations, rentals, pickup/return inspections, fuel readings, damages, rate plans, invoices and payments.
- Prevents double-booking of the same vehicle for overlapping periods.
- Ensures only customers with a valid (non-expired) driving licence can rent a vehicle.
- Guarantees a vehicle's return date/time is always after its pickup date/time, and that odometer/fuel readings are logically consistent.
- Automatically computes rental charges, extra-kilometre charges, late fees, damage charges and tax to generate an invoice, and never allows a negative charge or payment.

- Provides a simple Python front-end for staff to search availability, create reservations, issue and receive vehicles, and view management reports.

4. OBJECTIVES

1. To identify the entities, attributes and relationships of a car-rental business through requirement analysis.
2. To design a conceptual Entity-Relationship (ER) diagram covering all principal entities.
3. To convert the ER diagram into a normalized relational schema (up to 3NF) with a data dictionary.
4. To implement the schema in MySQL with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, DEFAULT constraints and triggers for cross-row business rules.
5. To populate the database with realistic sample data.
6. To write SQL queries using joins, aggregate functions, nested (sub) queries and views for the required management reports.
7. To build a Python (Tkinter) application that performs CRUD operations, input validation, and generates the required reports, connected to the database via MySQL Connector/Python.
8. To test the system for data integrity, error handling and correct report output.

5. EXISTING SYSTEM

Most small and medium car-rental agencies still use one or more of the following: paper registers for bookings, WhatsApp/phone confirmation of vehicle availability, and Excel sheets maintained separately by each branch for billing. These approaches share several weaknesses:

- No real-time check for whether a vehicle is already booked for the requested dates, leading to double-booking.
- No systematic record of vehicle condition at pickup and return, causing disputes over pre-existing damage.
- Manual, error-prone calculation of rental charges, extra-kilometre charges and late fees.
- Difficulty consolidating revenue and utilization figures across branches for management decisions.
- No enforcement of basic rules such as 'do not rent to a customer with an expired licence'.

6. PROPOSED SYSTEM

The proposed system replaces manual registers with a centralized, normalized MySQL database accessed through a Python desktop application. Every business rule that can be expressed declaratively is enforced by the database itself (primary keys, foreign keys, UNIQUE, CHECK, DEFAULT), and rules that need to compare across multiple rows (no overlapping rental, valid licence at pickup time, automatic vehicle-status change) are enforced with triggers, so that the rule holds no matter which part of the application — or which future application — writes to the database. The Python front-end offers a simple tabbed interface for

vehicle and customer master maintenance, availability search and reservation, vehicle issue and return with inspection, and one-click management reports.

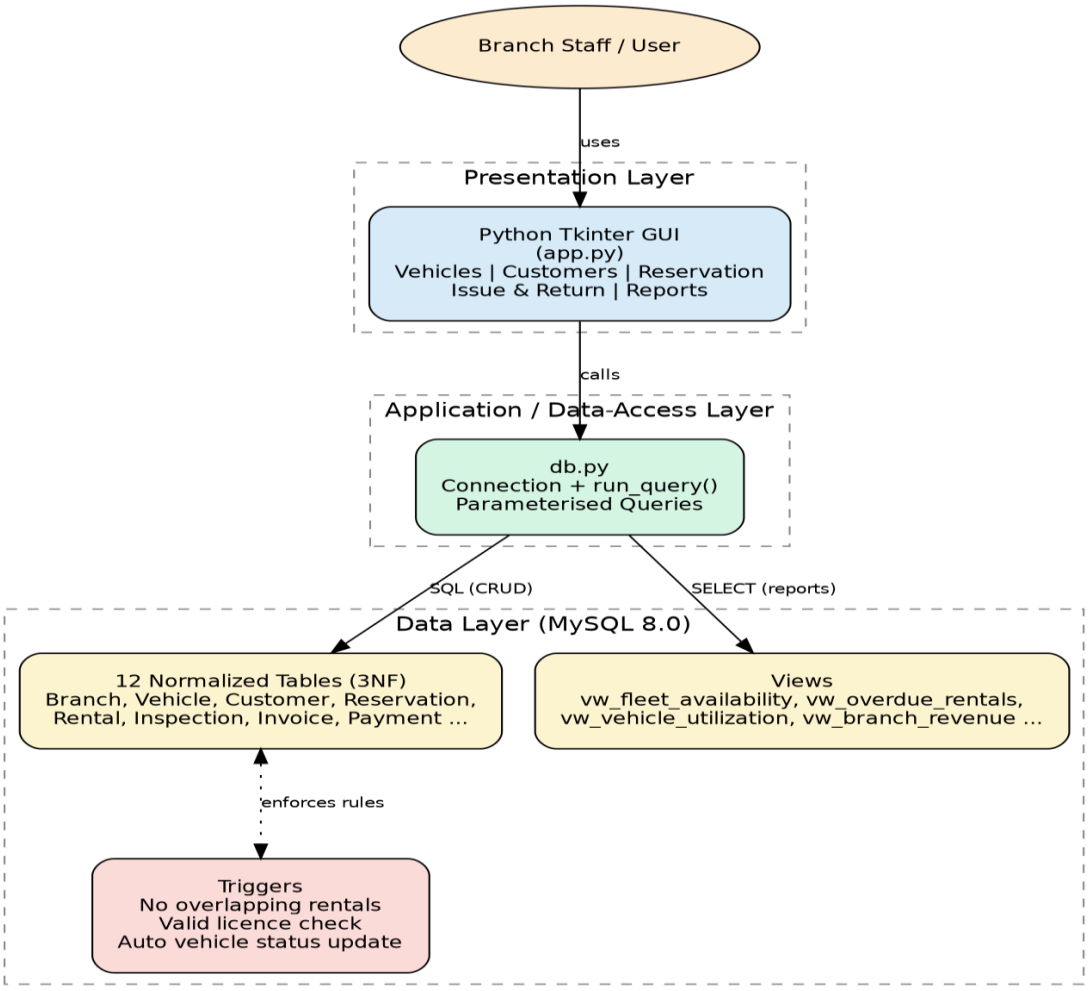


Figure 6.1: Three-tier architecture of the proposed system

7. METHODOLOGY

The project followed the standard database application development life-cycle:

1. Requirement gathering — identifying the users (branch staff), the data they need, and the rules that must always hold.
2. Conceptual design — drawing an Entity-Relationship diagram of all identified entities and relationships.
3. Logical design and normalization — converting the ER diagram into relational tables and checking each table against 1NF, 2NF and 3NF.
4. Physical design — writing DDL scripts with data types, keys, constraints, indexes and triggers in MySQL.
5. Data population — inserting realistic sample data for testing.
6. Query design — writing SELECT queries with joins, aggregates, subqueries, and packaging the most useful ones as views.
7. Application development — building the Python Tkinter front-end, connected via a small, reusable database-access module.

8. Testing — verifying CRUD operations, constraint violations, trigger behaviour and report correctness.

8. SYSTEM / ARCHITECTURE DESIGN

8.1 Entity-Relationship Diagram

The conceptual design below shows the twelve entities of the system and their relationships. Crow's-foot notation is used: a crow's foot means 'many'.

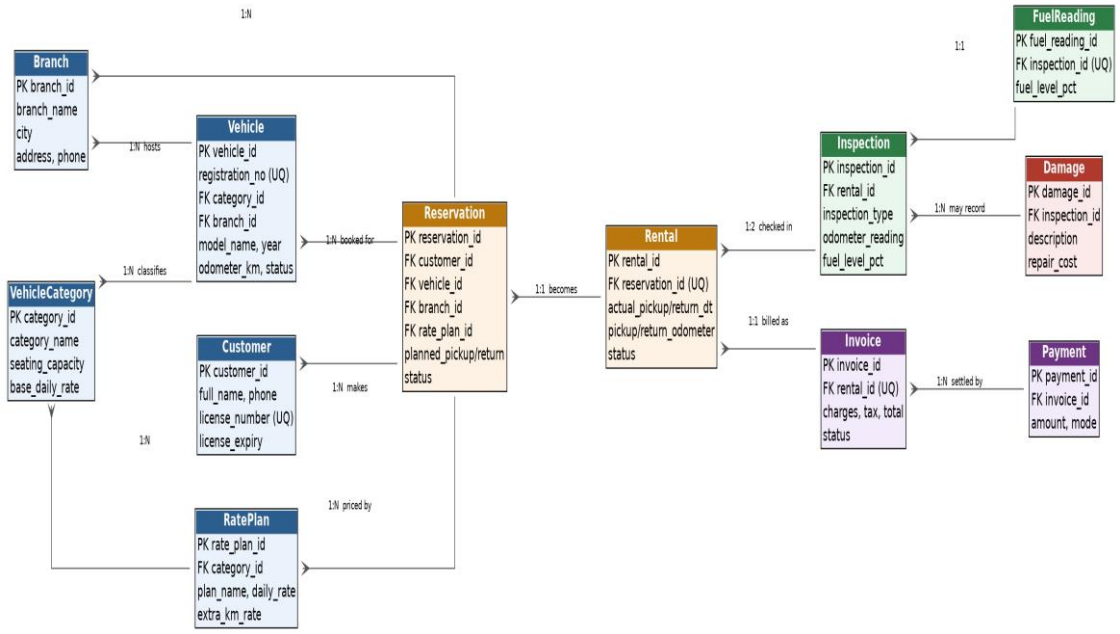


Figure 8.1: Entity-Relationship Diagram (ERD)

8.2 Functional Dependencies and Normalization (up to 3NF)

Below are representative functional dependencies used to verify normalization for the key tables. Every non-key attribute depends on the whole primary key (2NF) and only on the primary key, not on another non-key attribute (3NF).

Table	Functional Dependency	Normal Form Check
Vehicle	vehicle_id → registration_no, category_id, branch_id, model_name, year, odometer_km, status	All attributes depend only on vehicle_id (single-column PK) → 3NF satisfied
Reservation	reservation_id → customer_id, vehicle_id, branch_id, rate_plan_id, planned_pickup, planned_return, status	No partial or transitive dependency → 3NF satisfied
Invoice	invoice_id → rental_id, rental_charges, extra_km_charges, late_fee, damage_charges, tax_amount, total_amount, status	total_amount is derived/stored for reporting speed but functionally depends only on invoice_id → 3NF satisfied
RatePlan	rate_plan_id → category_id, plan_name, daily_rate, extra_km_rate, late_fee_per_hour	No transitive dependency through category_id → 3NF satisfied

Design decisions made during normalization:

- Vehicle category attributes (seating capacity, base rate) were separated into VehicleCategory instead of repeating them on every Vehicle row — removes a partial/transitive dependency and update anomaly.
- Inspection was separated from Rental because a rental has exactly two inspections (pickup and return) with different attribute values each time — storing them as extra columns on Rental would repeat the odometer/fuel/notes columns twice and violate 1NF (repeating groups).
- Invoice charges are broken into rental_charges, extra_km_charges, late_fee and damage_charges instead of one lump sum, so each can be audited and traced to its source rule.

8.3 Data Dictionary (Key Tables)

Table	Key Columns	Description
Branch	branch_id (PK)	Rental office locations
VehicleCategory	category_id (PK)	Hatchback / Sedan / SUV / Luxury, with base rate
Vehicle	vehicle_id (PK), category_id/branch_id (FK)	Individual vehicles in the fleet
Customer	customer_id (PK)	Renters, with driving-licence details
RatePlan	rate_plan_id (PK), category_id (FK)	Daily/extra-km/late-fee rates per category
Reservation	reservation_id (PK), customer_id/vehicle_id/branch_id/rate_plan_id (FK)	Advance booking of a vehicle
Rental	rental_id (PK), reservation_id (FK, UQ)	Actual pickup and return of the vehicle
Inspection	inspection_id (PK), rental_id (FK)	Condition check at pickup and return
FuelReading	fuel_reading_id (PK), inspection_id (FK, UQ)	Fuel level recorded at each inspection
Damage	damage_id (PK), inspection_id (FK)	Damage noted during an inspection
Invoice	invoice_id (PK), rental_id (FK, UQ)	Bill generated for a completed rental
Payment	payment_id (PK), invoice_id (FK)	Amount paid against an invoice

8.4 Business Rules and How They Are Enforced

Business Rule	Enforcement Mechanism
No overlapping rentals for the same vehicle	BEFORE INSERT trigger trg_no_overlap_reservation on Reservation
Valid (non-expired) driving licence	BEFORE INSERT trigger trg_valid_license on Reservation
Return date/time after pickup date/time	CHECK constraint chk_rental_dates on Rental

Valid odometer / fuel readings	CHECK constraints on Rental, Inspection, FuelReading (e.g. fuel 0–100, return_odometer ≥ pickup_odometer)
Non-negative charges and payments	CHECK constraints chk_invoice_amounts and chk_payment_amount
Unique registration number / licence number	UNIQUE constraints on Vehicle.registration_no and Customer.license_number
Automatic vehicle status change on pickup/return	AFTER UPDATE trigger trg_vehicle_status_on_pickup on Rental

8.5 Process Flow

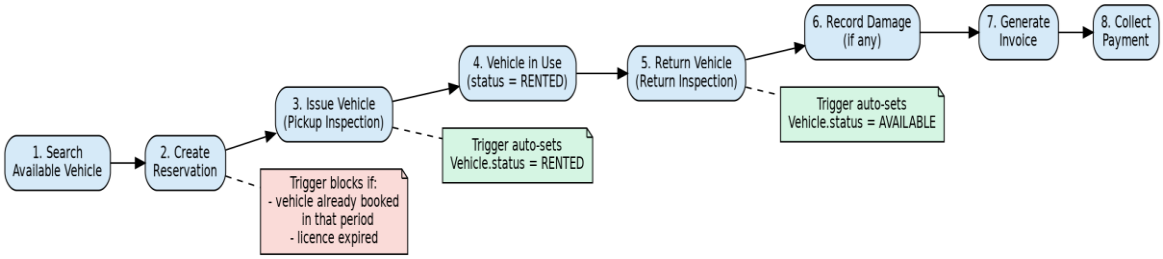


Figure 8.2: Reservation-to-Payment process flow

9. IMPLEMENTATION

9.1 Database Implementation

The database was implemented in MySQL 8.0 using three scripts, run in order:

- 01_schema.sql — creates the database, all 12 tables with primary/foreign keys and CHECK constraints, indexes on frequently searched columns (vehicle status, customer licence number, reservation dates), and the three business-rule triggers.
- 02_sample_data.sql — inserts realistic sample data: 3 branches, 4 vehicle categories, 6 vehicles, 4 customers, 4 rate plans, 3 reservations, 2 rentals (one completed, one active/overdue), inspections, a fuel reading set, a damage record, an invoice and a payment.
- 03_queries_and_views.sql — defines six reporting views (fleet availability, overdue rentals, vehicle utilization, damage report, branch revenue, customer history) plus three additional nested-subquery examples and an aggregate monthly-bookings query.

Sample DDL excerpt showing constraints on the Rental table:

```
CREATE TABLE Rental (
   rental_id INT AUTO_INCREMENT PRIMARY KEY,
   reservation_id INT NOT NULL UNIQUE,
   actual_pickup_dt DATETIME NULL,
   actual_return_dt DATETIME NULL,
   pickup_odometer INT NULL,
   return_odometer INT NULL,
   status ENUM('ACTIVE','RETURNED','OVERDUE') NOT NULL DEFAULT 'ACTIVE',
   CONSTRAINT fk_rental_reservation FOREIGN KEY (reservation_id)
      REFERENCES Reservation(reservation_id),
   CONSTRAINT chk_rental_dates CHECK
      (actual_return_dt IS NULL OR actual_return_dt > actual_pickup_dt),
   CONSTRAINT chk_rental_odometer CHECK
      (return_odometer IS NULL OR return_odometer >= pickup_odometer)
);99I
```

Customers, Reservation/Availability, Issue & Return, and Reports. All database access goes through a single helper module (db.py) whose run_query() function always uses parameterised queries (never string-formatted SQL), which prevents SQL-injection and keeps error handling in one place. When a trigger raises a business-rule violation (SIGNAL SQLSTATE '45000'), the helper's error message is caught and shown to the user as a friendly message box instead of a raw stack trace.

Key workflow implemented in code:

- Issue Vehicle: creates a Rental row, updates it with the pickup odometer/time (which fires the trigger that sets Vehicle.status = 'RENTED'), and records a PICKUP inspection with a fuel reading.
- Return Vehicle: updates the Rental row with return odometer/time (firing the trigger that sets Vehicle.status = 'AVAILABLE'), records a RETURN inspection and optional damage, then computes rental days × daily rate, extra-kilometre charges beyond a free allowance, late fee (if returned after the planned time), damage charges and 18% tax, and inserts the resulting Invoice.

- Reports tab: a drop-down lets the user run any of the six views directly and see the result in a table (Treeview).

10. RESULTS / OUTPUT

After running the three SQL scripts and the Python application, the following outputs were verified (see the Screenshots section of the submission checklist for the exact captures to include):

- All 12 tables created successfully with no DDL errors; SHOW TABLES lists all tables.
- Attempting to insert a reservation with overlapping dates for the same vehicle is rejected with the message 'Overlapping reservation exists for this vehicle in the selected period'.
- Attempting to insert a reservation for a customer whose licence has expired is rejected with 'Customer driving licence is expired before the pickup date'.
- Issuing a vehicle changes its status from AVAILABLE to RENTED automatically; returning it changes the status back to AVAILABLE automatically — confirming the trigger works without any application code doing the UPDATE on Vehicle directly.
- The Fleet Availability report correctly excludes vehicles currently RENTED or in MAINTENANCE.
- The Overdue Rentals report correctly lists the SUV rented by Ananya Rao, whose planned return time has passed with no actual return recorded.
- The generated invoice for the first rental correctly totals rental charges + damage charges + 18% tax.

11. ADVANTAGES

- Eliminates double-booking through a database trigger, not just application logic — the rule holds even if another application connects directly to the database.
- Single, consistent record of vehicle condition (inspections) removes disputes about pre-existing damage.
- Automatic, formula-based billing removes manual calculation errors.
- Centralized data across branches enables instant branch-wise revenue and utilization reporting.
- Referential integrity (foreign keys) prevents orphan records, e.g. an invoice for a rental that does not exist.
- Indexes on frequently searched columns (vehicle status, licence number, reservation dates) keep availability search fast even as data grows.

12. LIMITATIONS

- The Tkinter interface is functional but not web-based, so it must be run on a machine with direct network access to the MySQL server.
- The application currently has a single implicit user role (branch staff); it does not yet implement separate login/roles for manager vs. front-desk staff.
- Damage-cost estimation is entered manually rather than through an integrated third-party repair-estimate service.

- Fuel level is recorded as a percentage entered by staff, not read automatically from a vehicle sensor.

13. FUTURE SCOPE

- Convert the desktop application into a web application (e.g., Flask/Django) so branch staff can use it from any browser.
- Add role-based login (Admin, Branch Manager, Front-desk Staff) with different permissions.
- Integrate SMS/email notifications for reservation confirmation and overdue-return reminders.
- Add a customer-facing online booking portal that reads from the same vw_fleet_availability view.
- Integrate a payment gateway for online advance payment at the time of reservation.

14. CONCLUSION

This project successfully designed and implemented a normalized (3NF) relational database for a Car Rental Reservation and Vehicle Return Management System, together with a working Python front-end. The design demonstrates how declarative constraints (primary/foreign keys, UNIQUE, CHECK) and triggers together enforce real business rules — no overlapping rentals, valid licences, non-negative charges, and automatic vehicle-status tracking — directly inside the database, so that data remains correct regardless of which application accesses it. The reporting views (fleet availability, overdue rentals, utilization, damages, branch revenue, customer history) show how well-designed SQL queries directly support day-to-day management decisions. The project meets every deliverable specified for Review 1, Review 2 and Review 3 of the course's Common Review Framework.

15. REFERENCES

- Elmasri, R. and Navathe, S.B., Fundamentals of Database Systems, Pearson Education.
- Silberschatz, A., Korth, H.F., and Sudarshan, S., Database System Concepts, McGraw-Hill.
- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
- Python 3 Documentation, Tkinter — https://docs.python.org/3/library/tkinter.html
- MySQL Connector/Python Developer Guide — https://dev.mysql.com/doc/connector-python/en/